Homework 4

TJ Raklovits

13.11.2025

Contents

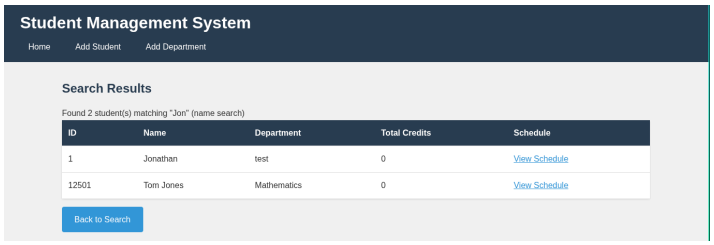
Link to Code 1
Requirement 1 1
Requirement 2 1
Requirement 3 2
Requirement 4 2
CODE 3
`app.py` 3
`config.py` 8
`database.py` 8
`static/style.css` 9
`templates/add_department.html` 14
`templates/add_student.html` 15
`templates/base.html` 16
`templates/index.html` 17
`templates/schedule.html` 18
`templates/search_results.html` 19

Link to Code

github.com/monster0506/databaseshomework4

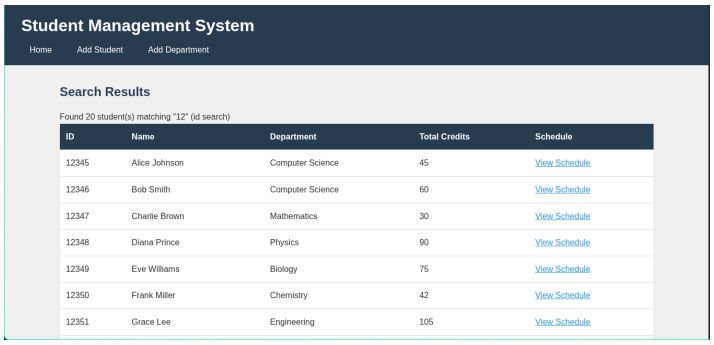
Requirement 1

```sql
SELECT id, name, dept_name, tot_cred
  FROM learners
  WHERE name LIKE %s
  ORDER BY name
```



Requirement 2

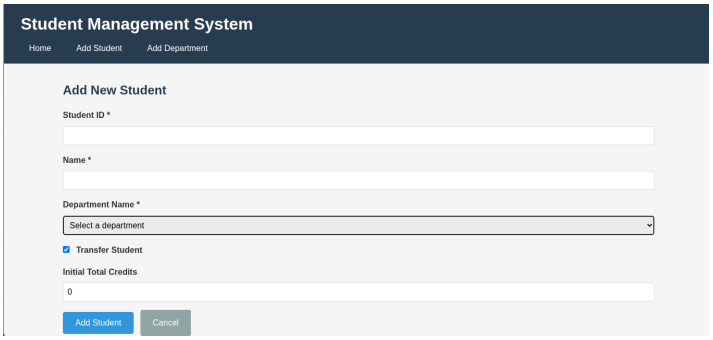
```sql
SELECT id, name, dept_name, tot_cred
  FROM learners
  WHERE CAST(id AS CHAR) LIKE %s
  ORDER BY id
```



Requirement 3

Note that the initial credits box is only editable when the `Transfer Student` checkbox is checked. Additionally, note that the departments dropdown is auto-populated to include current departments.

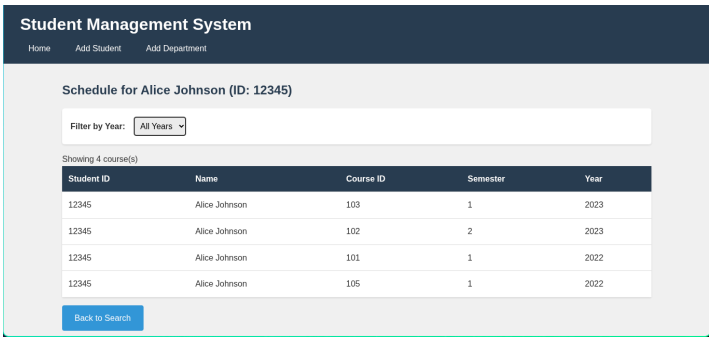
```python
sql = """
    INSERT INTO learners (id, name, dept_name, tot_cred)
    VALUES (%s, %s, %s, %s)
"""
cursor.execute(sql, (student_id_int, name, dept_name, tot_cred))
conn.commit()
```



Requirement 4

Note that the year dropdown is populated by existing years from the current schedule.

```python
sql = """
  SELECT l.id, l.name, r.course_id, r.semester, r.year
    FROM learners l
    JOIN registrations r ON l.id = r.id
      WHERE l.id = %s
    ORDER BY r.year DESC, r.semester, r.course_id
"""
cursor.execute(sql, (student_id_int,))
```



CODE

app.py

```python
from flask import Flask, flash, redirect, render_template, request, url_for

from database import get_connection


app = Flask(__name__)
app.secret_key = "SeCrEtKeY"


def get_departments():
    conn = get_connection()
    try:
        with conn.cursor() as cursor:
            cursor.execute(
                "SELECT DISTINCT dept_name FROM faculties ORDER BY dept_name"
            )
            depts = [row["dept_name"] for row in cursor.fetchall()]
            return depts
    finally:
        conn.close()


@app.route("/")
def index():

    return render_template("index.html")


@app.route("/search", methods=["GET"])
def search():

    search_type = request.args.get("type", "name")
    query = request.args.get("query", "").strip()

    if not query:
        flash("Please enter a search term.", "warning")
        return redirect(url_for("index"))

    conn = get_connection()
    try:
        with conn.cursor() as cursor:
            if search_type == "name":

                sql = """
                    SELECT id, name, dept_name, tot_cred
                    FROM learners
                    WHERE name LIKE %s
                    ORDER BY name
                """
                pattern = f"%{query}%"
                cursor.execute(sql, (pattern,))
            else:
```

```python

                sql = """
                    SELECT id, name, dept_name, tot_cred
                    FROM learners
                    WHERE CAST(id AS CHAR) LIKE %s
                    ORDER BY id
                """
                pattern = f"%{query}%"
                cursor.execute(sql, (pattern,))

            results = cursor.fetchall()

            return render_template(
                "search_results.html",
                students=results,
                search_type=search_type,
                query=query,
            )
    finally:
        conn.close()


@app.route("/add_student", methods=["GET", "POST"])
def add_student():

    if request.method == "POST":
        student_id = request.form.get("student_id", "").strip()
        name = request.form.get("name", "").strip()
        dept_name = request.form.get("dept_name", "").strip()
        is_transfer = request.form.get("is_transfer") == "on"
        tot_cred = request.form.get("tot_cred", "0").strip()

        if not student_id or not name or not dept_name:
            flash("All fields are required.", "error")
            return redirect(url_for("add_student"))

        if is_transfer:
            try:
                tot_cred = int(tot_cred)
            except ValueError:
                flash("Invalid credit value for transfer student.", "error")
                return redirect(url_for("add_student"))
        else:
            tot_cred = 0

        conn = get_connection()
        try:
            with conn.cursor() as cursor:

                try:
                    student_id_int = int(student_id)
                except ValueError:
                    flash("Student ID must be a number.", "error")
                    return redirect(url_for("add_student"))

```

```
                cursor.execute(
                    "SELECT id FROM learners WHERE id = %s", (student_id_int,)
                )
                if cursor.fetchone():
                    flash(f"Student ID {student_id} already exists.", "error")
                    return redirect(url_for("add_student"))

                sql = """
                    INSERT INTO learners (id, name, dept_name, tot_cred)
                    VALUES (%s, %s, %s, %s)
                """
                cursor.execute(sql, (student_id_int, name, dept_name, tot_cred))
                conn.commit()

                flash(
                    f"Student {name} (ID: {student_id}) added successfully!", "success"
                )
                return redirect(url_for("index"))
        except Exception as e:
            conn.rollback()
            flash(f"Error adding student: {str(e)}", "error")
            return redirect(url_for("add_student"))
        finally:
            conn.close()
    else:

        departments = get_departments()
        return render_template("add_student.html", departments=departments)


@app.route("/add_department", methods=["GET", "POST"])
def add_department():
    if request.method == "POST":
        dept_name = request.form.get("dept_name", "").strip()
        building = request.form.get("building", "").strip()
        budget = request.form.get("budget", "").strip()

        if not dept_name:
            flash("Department name is required.", "error")
            return redirect(url_for("add_department"))

        try:
            budget_float = float(budget) if budget else 0.0
        except ValueError:
            flash("Budget must be a valid number.", "error")
            return redirect(url_for("add_department"))

        conn = get_connection()
        try:
            with conn.cursor() as cursor:

                cursor.execute(
                    "SELECT dept_name FROM faculties WHERE dept_name = %s", (dept_name,)
                )
                if cursor.fetchone():
```

```
                    flash(f"Department {dept_name} already exists.", "error")
                    return redirect(url_for("add_department"))

                sql = """
                    INSERT INTO faculties (dept_name, building, budget)
                    VALUES (%s, %s, %s)
                """
                cursor.execute(
                    sql, (dept_name, building if building else None, budget_float)
                )
                conn.commit()

                flash(f"Department {dept_name} added successfully!", "success")
                return redirect(url_for("index"))
        except Exception as e:
            conn.rollback()
            flash(f"Error adding department: {str(e)}", "error")
            return redirect(url_for("add_department"))
        finally:
            conn.close()
    else:

        return render_template("add_department.html")


@app.route("/schedule/<student_id>")
def schedule(student_id):

    year_filter = request.args.get("year", "").strip()

    conn = get_connection()
    try:
        with conn.cursor() as cursor:

            try:
                student_id_int = int(student_id)
            except ValueError:
                flash(f"Invalid student ID: {student_id}", "error")
                return redirect(url_for("index"))

            cursor.execute(
                "SELECT id, name FROM learners WHERE id = %s", (student_id_int,)
            )
            student = cursor.fetchone()

            if not student:
                flash(f"Student with ID {student_id} not found.", "error")
                return redirect(url_for("index"))

            if year_filter:

                try:
                    year_filter_int = int(year_filter)
                except ValueError:
                    flash(f"Invalid year filter: {year_filter}", "error")
```

```
                    return redirect(url_for("schedule", student_id=student_id))

                sql = """
                    SELECT l.id, l.name, r.course_id, r.semester, r.year
                    FROM learners l
                    JOIN registrations r ON l.id = r.id
                    WHERE l.id = %s AND r.year = %s
                    ORDER BY r.year DESC, r.semester, r.course_id
                """
                cursor.execute(sql, (student_id_int, year_filter_int))
            else:
                sql = """
                    SELECT l.id, l.name, r.course_id, r.semester, r.year
                    FROM learners l
                    JOIN registrations r ON l.id = r.id
                    WHERE l.id = %s
                    ORDER BY r.year DESC, r.semester, r.course_id
                """
                cursor.execute(sql, (student_id_int,))

            schedule_data = cursor.fetchall()

            sql_years = """
                SELECT DISTINCT year
                FROM registrations
                WHERE id = %s
                ORDER BY year DESC
            """
            cursor.execute(sql_years, (student_id_int,))
            available_years = [row["year"] for row in cursor.fetchall()]

            return render_template(
                "schedule.html",
                student=student,
                schedule=schedule_data,
                available_years=available_years,
                selected_year=year_filter,
            )
    finally:
        conn.close()


if __name__ == "__main__":
    app.run(debug=True)
```

config.py

```python
import configparser
import os


def load_config(config_path="config.ini"):

    config = configparser.ConfigParser()

    if not os.path.exists(config_path):
        raise FileNotFoundError(
            f"Config file '{config_path}' not found. "
            "Please create it with database credentials."
        )

    config.read(config_path)

    if "database" not in config:
        raise ValueError("Config file must contain a [database] section.")

    db_config = config["database"]

    required_keys = ["host", "user", "password", "database"]
    missing_keys = [key for key in required_keys if key not in db_config]

    if missing_keys:
        raise ValueError(
            f"Config file missing required keys: {', '.join(missing_keys)}"
        )

    return {
        "host": db_config["host"],
        "user": db_config["user"],
        "password": db_config["password"],
        "database": db_config["database"],
        "charset": "utf8mb4",
        "cursorclass": None,
    }
```

database.py

```python
import pymysql

from config import load_config


def get_connection():

    config = load_config()
    return pymysql.connect(
        host=config["host"],
        user=config["user"],
        password=config["password"],
        database=config["database"],
        charset=config["charset"],
```

```python
        cursorclass=pymysql.cursors.DictCursor,
    )


def discover_schema():

    conn = get_connection()
    try:
        with conn.cursor() as cursor:

            cursor.execute("SHOW TABLES")
            tables = [
                row[f'Tables_in_{load_config()["database"]}']
                for row in cursor.fetchall()
            ]

            if "student" in tables:
                cursor.execute("DESCRIBE student")
                student_columns = [row["Field"] for row in cursor.fetchall()]

            dept_table = None
            if "department" in tables:
                dept_table = "department"
            elif "departments" in tables:
                dept_table = "departments"

            takes_table = None
            if "takes" in tables:
                takes_table = "takes"
            elif "enrollment" in tables:
                takes_table = "enrollment"
            elif "enrollments" in tables:
                takes_table = "enrollments"

            return {
                "tables": tables,
                "student_columns": student_columns if "student" in tables else [],
                "department_table": dept_table,
                "takes_table": takes_table,
            }
    finally:
        conn.close()
```

static/style.css

```css
* {
    margin: 0;
    padding: 0;
    box-sizing: border-box;
}

body {
    font-family: Arial, sans-serif;
    line-height: 1.6;
    color: #333;
```

```
    background-color: #f4f4f4;
}

nav {
    background-color: #2c3e50;
    color: white;
    padding: 1rem 2rem;
    margin-bottom: 2rem;
}

nav h1 {
    margin-bottom: 0.5rem;
}

nav ul {
    list-style: none;
    display: flex;
    gap: 1rem;
}

nav a {
    color: white;
    text-decoration: none;
    padding: 0.5rem 1rem;
    border-radius: 4px;
    transition: background-color 0.3s;
}

nav a:hover {
    background-color: #34495e;
}

main {
    max-width: 1200px;
    margin: 0 auto;
    padding: 0 2rem;
}

.messages {
    margin-bottom: 1rem;
}

.alert {
    padding: 1rem;
    margin-bottom: 0.5rem;
    border-radius: 4px;
}

.alert-success {
    background-color: #d4edda;
    color: #155724;
    border: 1px solid #c3e6cb;
}

.alert-error {
```

```
    background-color: #f8d7da;
    color: #721c24;
    border: 1px solid #f5c6cb;
}

.alert-warning {
    background-color: #fff3cd;
    color: #856404;
    border: 1px solid #ffeeba;
}

h2 {
    margin-bottom: 1rem;
    color: #2c3e50;
}

.search-forms {
    display: grid;
    grid-template-columns: 1fr 1fr;
    gap: 2rem;
    margin-bottom: 2rem;
}

.search-form {
    background: white;
    padding: 1.5rem;
    border-radius: 8px;
    box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

.search-form h3 {
    margin-bottom: 1rem;
    color: #2c3e50;
}

.form-group {
    margin-bottom: 1rem;
}

.form-group label {
    display: block;
    margin-bottom: 0.5rem;
    font-weight: bold;
}

.form-group input[type="text"],
.form-group input[type="number"],
.form-group select {
    width: 100%;
    padding: 0.5rem;
    border: 1px solid #ddd;
    border-radius: 4px;
    font-size: 1rem;
}

```

```
.form-group input[type="checkbox"] {
    margin-right: 0.5rem;
}

button, .btn {
    padding: 0.75rem 1.5rem;
    background-color: #3498db;
    color: white;
    border: none;
    border-radius: 4px;
    cursor: pointer;
    text-decoration: none;
    display: inline-block;
    font-size: 1rem;
    margin-right: 0.5rem;
    transition: background-color 0.3s;
}

button:hover, .btn:hover {
    background-color: #2980b9;
}

.btn-secondary {
    background-color: #95a5a6;
}

.btn-secondary:hover {
    background-color: #7f8c8d;
}

.btn-link {
    color: #3498db;
    text-decoration: underline;
}

.btn-link:hover {
    color: #2980b9;
}

table {
    width: 100%;
    border-collapse: collapse;
    background: white;
    margin-bottom: 1rem;
    box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

table thead {
    background-color: #2c3e50;
    color: white;
}

table th,
table td {
    padding: 0.75rem;
```

```
    text-align: left;
    border-bottom: 1px solid #ddd;
}

table tbody tr:hover {
    background-color: #f5f5f5;
}

table tbody tr:last-child td {
    border-bottom: none;
}

.year-filter {
    background: white;
    padding: 1rem;
    border-radius: 4px;
    margin-bottom: 1rem;
    box-shadow: 0 2px 4px rgba(0,0,0,0.1);
}

.year-filter form {
    display: flex;
    align-items: center;
    gap: 1rem;
}

.year-filter label {
    font-weight: bold;
}

.year-filter select {
    padding: 0.5rem;
    border: 1px solid #ddd;
    border-radius: 4px;
    font-size: 1rem;
}

@media (max-width: 768px) {
    .search-forms {
        grid-template-columns: 1fr;
    }

    nav ul {
        flex-direction: column;
    }
}
```

templates/add_department.html

```html
{% extends "base.html" %}

{% block title %}Add Department - Student Management{% endblock %}

{% block content %}
<h2>Add New Department</h2>

<form action="{{ url_for('add_department') }}" method="POST">
    <div class="form-group">
        <label for="dept_name">Department Name *</label>
        <input type="text" id="dept_name" name="dept_name" required>
    </div>

    <div class="form-group">
        <label for="building">Building</label>
        <input type="text" id="building" name="building" placeholder="e.g., Taylor Hall">
    </div>

    <div class="form-group">
        <label for="budget">Budget</label>
        <input type="number" id="budget" name="budget" step="0.01" min="0" placeholder="0.00">
    </div>

    <button type="submit">Add Department</button>
    <a href="{{ url_for('index') }}" class="btn btn-secondary">Cancel</a>
</form>
{% endblock %}

{% extends "base.html" %}
```

templates/add_student.html

```html
{% block title %}Add Student - Student Management{% endblock %}

{% block content %}
<h2>Add New Student</h2>

<form action="{{ url_for('add_student') }}" method="POST">
    <div class="form-group">
        <label for="student_id">Student ID *</label>
        <input type="text" id="student_id" name="student_id" required>
    </div>

    <div class="form-group">
        <label for="name">Name *</label>
        <input type="text" id="name" name="name" required>
    </div>

    <div class="form-group">
        <label for="dept_name">Department Name *</label>
        <select id="dept_name" name="dept_name" required>
            <option value="">Select a department</option>
            {% for dept in departments %}
            <option value="{{ dept }}">{{ dept }}</option>
            {% endfor %}
        </select>
    </div>

    <div class="form-group">
        <label>
            <input type="checkbox" id="is_transfer" name="is_transfer" onchange="toggleTransferCredits()">
            Transfer Student
        </label>
    </div>

    <div class="form-group" id="transfer_credits_group" style="display: none;">
        <label for="tot_cred">Initial Total Credits</label>
        <input type="number" id="tot_cred" name="tot_cred" min="0" value="0">
    </div>

    <button type="submit">Add Student</button><a href="{{ url_for('index') }}" class="btn btn-secondary">Cancel</a>
</form>

<script>
    function toggleTransferCredits() {
        const checkbox = document.getElementById('is_transfer');
        const creditsGroup = document.getElementById('transfer_credits_group');
        if (checkbox.checked) { creditsGroup.style.display = 'block';
        } else {
            creditsGroup.style.display = 'none';
            document.getElementById('tot_cred').value = '0';
        }
    }
</script>
{% endblock %}
```

templates/base.html

```html
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1.0">
        <title>{% block title %}Student Management{% endblock %}</title>
        <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
    </head>
    <body>
        <nav>
            <h1>Student Management System</h1>
            <ul>
                <li><a href="{{ url_for('index') }}">Home</a></li>
                <li><a href="{{ url_for('add_student') }}">Add Student</a></li>
                <li><a href="{{ url_for('add_department') }}">Add Department</a></li>
            </ul>
        </nav>

        <main>
            {% with messages = get_flashed_messages(with_categories=true) %}
            {% if messages %}
            <div class="messages">
                {% for category, message in messages %}
                <div class="alert alert-{{ category }}">{{ message }}</div>
                {% endfor %}
            </div>
            {% endif %}
            {% endwith %}

            {% block content %}{% endblock %}
        </main>
    </body>
</html>
```

templates/index.html

```html
{% extends "base.html" %}

{% block title %}Student Search - Student Management{% endblock %}

{% block content %}
<h2>Search for Students</h2>

<div class="search-forms">
    <div class="search-form">
        <h3>Search by Name</h3>
        <form action="{{ url_for('search') }}" method="GET">
            <input type="hidden" name="type" value="name">
            <div class="form-group">
                <label for="query_name">Enter student name (partial match):</label>
                <input type="text" id="query_name" name="query" required placeholder="e.g., Jon">
            </div>
            <button type="submit">Search</button>
        </form>
    </div>

    <div class="search-form">
        <h3>Search by ID</h3>
        <form action="{{ url_for('search') }}" method="GET">
            <input type="hidden" name="type" value="id">
            <div class="form-group">
                <label for="query_id">Enter student ID (partial match):</label>
                <input type="text" id="query_id" name="query" required placeholder="e.g., 12">
            </div>
            <button type="submit">Search</button>
        </form>
    </div>
</div>
{% endblock %}
```

templates/schedule.html

```html
{% extends "base.html" %}

{% block title %}{{ student.name }}'s Schedule - Student Management{% endblock %}

{% block content %}
<h2>Schedule for {{ student.name }} (ID: {{ student.id }})</h2>

{% if available_years %}
<div class="year-filter">
    <form method="GET" action="{{ url_for('schedule', student_id=student.id) }}">
        <label for="year">Filter by Year:</label>
        <select id="year" name="year" onchange="this.form.submit()">
            <option value="">All Years</option>
            {% for year in available_years %}
            <option value="{{ year }}" {% if selected_year == year|string %}selected{% endif %}>
            {{ year }}
            </option>
            {% endfor %}
        </select>
    </form>
</div>
{% endif %}

{% if schedule %}
<p>Showing {{ schedule|length }} course(s)</p>

<table>
    <thead>
        <tr>
            <th>Student ID</th>
            <th>Name</th>
            <th>Course ID</th>
            <th>Semester</th>
            <th>Year</th>
        </tr>
    </thead>
    <tbody>
        {% for course in schedule %}
        <tr>
            <td>{{ course.id }}</td>
            <td>{{ course.name }}</td>
            <td>{{ course.course_id }}</td>
            <td>{{ course.semester }}</td>
            <td>{{ course.year }}</td>
        </tr>
        {% endfor %}
    </tbody>
</table>
{% else %}
<p>No courses found for this student.</p>
{% endif %}

<a href="{{ url_for('index') }}" class="btn">Back to Search</a>
{% endblock %}
```

templates/search_results.html

```html
{% extends "base.html" %}

{% block title %}Search Results - Student Management{% endblock %}

{% block content %}
<h2>Search Results</h2>

{% if students %}
<p>Found {{ students|length }} student(s) matching "{{ query }}" ({{ search_type }} search)</p>

<table>
    <thead>
        <tr>
            <th>ID</th>
            <th>Name</th>
            <th>Department</th>
            <th>Total Credits</th>
            <th>Schedule</th>
        </tr>
    </thead>
    <tbody>
        {% for student in students %}
        <tr>
            <td>{{ student.id }}</td>
            <td>{{ student.name }}</td>
            <td>{{ student.dept_name or 'N/A' }}</td>
            <td>{{ student.tot_cred }}</td>
            <td>
                <a href="{{ url_for('schedule', student_id=student.id) }}" class="btn-link">View Schedule</a>
            </td>
        </tr>
        {% endfor %}
    </tbody>
</table>
{% else %}
<p>No students found matching "{{ query }}" ({{ search_type }} search).</p>
{% endif %}

<a href="{{ url_for('index') }}" class="btn">Back to Search</a>
{% endblock %}
```